

PROMPT — Conectar PostgreSQL real y endpoints de operaciones grupales

Pegar completo en Replit

Hay dos tareas críticas pendientes que debo resolver. Por favor hacerlas en orden.

TAREA 1 — Conectar el sistema a PostgreSQL real

Actualmente `server/routes.ts` importa `storage` desde `./storage` (MemStorage — datos en memoria). Ya existe `server/db-storage.ts` implementado con queries reales de Drizzle/PostgreSQL pero no se está usando. Hay que hacer el cambio.

Paso 1 — Cambiar el import en `server/routes.ts` :

Buscar esta línea:

```
import { storage } from './storage';
```

Reemplazarla por:

```
import { storage } from './db-storage';
```

Paso 2 — Verificar que todos los métodos existen en `db-storage.ts`

Después del cambio de import, el servidor puede tirar errores si algún método que usa `routes.ts` no está implementado en `db-storage.ts` . Por cada error de compilación o runtime que aparezca:

- Identificar qué método falta en `db-storage.ts`
- Implementarlo siguiendo el mismo patrón Drizzle que ya tienen los otros métodos del archivo

El patrón estándar es:

```
// GET todos
```

```

async getNombreItems(): Promise<TipoItem[]> {
  return await db.select().from(schema.nombreTabla);
}

// GET por ID
async getNombreItem(id: string): Promise<TipoItem | undefined> {
  const result = await db.select().from(schema.nombreTabla).where(eq(schema.nombr
  return result[0];
}

// CREATE
async createNombreItem(item: InsertTipoItem): Promise<TipoItem> {
  const result = await db.insert(schema.nombreTabla).values({ id: randomUUID(), .
  return result[0];
}

// UPDATE
async updateNombreItem(id: string, item: Partial<InsertTipoItem>): Promise<TipoIt
  const result = await db.update(schema.nombreTabla).set(item).where(eq(schema.no
  return result[0];
}

// DELETE
async deleteNombreItem(id: string): Promise<boolean> {
  const result = await db.delete(schema.nombreTabla).where(eq(schema.nombreTabla.
  return result.length > 0;
}

```

Paso 3 — Ejecutar `npm run db:push`

Verificar que todas las tablas del schema estén creadas en PostgreSQL. Si da error, resolverlo antes de continuar.

Paso 4 — Verificar que el sistema funciona

Probar que estos endpoints responden con datos reales (no vacíos ni errores 500):

- GET /api/rooms
- GET /api/reservations
- GET /api/guests
- GET /api/dashboard/stats

Si alguno falla, revisar el método correspondiente en `db-storage.ts` y corregirlo.

Paso 5 — Ejecutar el seed

Una vez que la conexión a PostgreSQL funciona, ejecutar `server/seed.ts` para cargar los datos de ejemplo en la base de datos real. Verificar que después de correr el seed los endpoints devuelven datos.

TAREA 2 — Implementar endpoints de operaciones grupales

Los botones de Check-in Grupal, Check-out Grupal, Pago Grupal y Factura Grupal ya están implementados en el frontend (`client/src/pages/group-detail.tsx`) pero les faltan los endpoints del backend. Implementar los siguientes 4 endpoints en `server/routes.ts` y sus métodos en `server/db-storage.ts` .

Endpoint 1 — Check-in Grupal

`POST /api/groups/:groupId/bulk-checkin`

Lógica:

1. Obtener todas las reservas del grupo que estén en estado `confirmed`
2. Para cada reserva, verificar el estado de su habitación asignada (`roomId`)
3. Si la habitación está en estado `available` :
 - Cambiar estado de la reserva a `checked_in`
 - Cambiar estado de la habitación a `occupied`
4. Si la habitación NO está en estado `available` (está en `cleaning` , `maintenance` , etc.): omitir esa reserva sin error
5. Registrar la acción en `auditLogs`

Retornar:

```
{
  "processed": 5,
  "skipped": 2,
  "skippedRooms": ["301", "402"]
}
```

Método en `db-storage.ts` :

```
async bulkCheckIn(groupId: string): Promise<{ processed: number; skipped: number;
```

Endpoint 2 — Check-out Grupal

```
POST /api/groups/:groupId/bulk-checkout
```

Lógica:

1. Obtener todas las reservas del grupo en estado `checked_in`
2. Para cada reserva, calcular el saldo pendiente: suma de cargos - suma de pagos
3. Si el saldo es 0:
 - Cambiar estado de la reserva a `checked_out`
 - Cambiar estado de la habitación a `cleaning`
 - Crear tarea de housekeeping: `type: "checkout_clean", priority: "high", status: "pending", con notes: "Check-out grupal — [nombre del grupo]"`
4. Si el saldo es mayor a 0: omitir esa reserva e incluirla en la lista de pendientes
5. Registrar en `auditLogs`

Retornar:

```
{
  "processed": 4,
  "skipped": 2,
  "pendingBalance": [
    { "room": "301", "guestName": "García Juan", "balance": 5400 },
    { "room": "402", "guestName": "López María", "balance": 12800 }
  ]
}
```

Método en `db-storage.ts` :

```
async bulkCheckOut(groupId: string): Promise<{ processed: number; skipped: number
```

Endpoint 3 — Pago Grupal

```
POST /api/groups/:groupId/bulk-payment
Body: {
  amount: number,
  method: string,
  receiptType: string,
  distribution: "equal" | "proportional",
  notes?: string
}
```

Lógica:

Obtener todas las reservas activas del grupo (estado `confirmed` o `checked_in`).

Si `distribution = "equal"`:

- Dividir el monto total en partes iguales entre todas las reservas activas
- La última reserva absorbe el redondeo (centavos sobrantes)

Si `distribution = "proportional"`:

- Calcular el saldo pendiente de cada reserva (suma cargos - suma pagos)
- Calcular el total de saldos pendientes del grupo
- Cada reserva recibe: $(\text{su saldo} / \text{total saldos}) * \text{monto total}$
- Redondear a 2 decimales, la última absorbe la diferencia

Para cada reserva, crear un pago usando el método `createPayment` con:

- `reservationId`: id de la reserva
- `amount`: monto calculado para esa reserva
- `method`: el método de pago recibido
- `receiptType`: el tipo de comprobante recibido
- `notes`: las notas recibidas + " (Pago grupal)"

Retornar:

```
{
  "success": true,
  "paymentsCreated": 6,
```

```
"distribution": [  
  { "room": "301", "guestName": "García Juan", "amount": 8500 },  
  { "room": "302", "guestName": "López María", "amount": 8500 }  
]  
}
```

Método en `db-storage.ts` :

```
async bulkPayment(groupId: string, amount: number, method: string, receiptType: s
```

Endpoint 4 — Resumen de Factura Grupal

```
GET /api/groups/:groupId/invoice-summary
```

Lógica:

1. Obtener todas las reservas del grupo con sus detalles
2. Para cada reserva obtener: lista de cargos, lista de pagos, saldo calculado
3. Calcular totales consolidados del grupo

Retornar:

```
{  
  "group": {  
    "id": "...",  
    "code": "GRP-0001",  
    "name": "Congreso AACR 2026",  
    "contactName": "...",  
    "checkInDate": "...",  
    "checkOutDate": "..."  
  },  
  "reservations": [  
    {  
      "id": "...",  
      "roomNumber": "301",  
      "roomType": "Suite Junior",  
      "guestName": "García Juan",  
      "checkIn": "2026-03-10",  
      "checkOut": "2026-03-13",  
      "nights": 3,  
      "status": "checked_in",  
      "charges": [...],  
    }  
  ]  
}
```

```
        "payments": [...],
        "totalCharges": 45000,
        "totalPayments": 20000,
        "balance": 25000
    }
],
"totals": {
    "totalAccommodation": 180000,
    "totalExtras": 35000,
    "totalCharges": 215000,
    "totalPayments": 80000,
    "totalBalance": 135000
}
}
```

Método en `db-storage.ts` :

```
async getGroupInvoiceSummary(groupId: string): Promise<GroupInvoiceSummary>
```

Definir el tipo `GroupInvoiceSummary` en `shared/schema.ts` o directamente en `db-storage.ts` si es solo para uso interno.

Verificación final

Después de completar ambas tareas:

1. Confirmar que el servidor corre sin errores
 2. Confirmar que `GET /api/rooms` devuelve datos de PostgreSQL (no vacío)
 3. Confirmar que los 4 nuevos endpoints responden correctamente con un grupo de prueba
 4. Confirmar que el proyecto compila sin errores TypeScript con `npm run check`
-

Fin del prompt